MA 5755: Data Analysis & Visualization in R/Python/SQL

Week 13: Neural Networks

Dr. Rakhi Singh
Department of Mathematics
IIT Madras

Spring 2026

Motivation: Function Approximation

We begin with the classical problem of approximating an unknown function

$$f^\star : \mathbb{R}^d \to \mathbb{R}$$

Given samples $\{(x_i, y_i)\}_{i=1}^n$ where $y_i \approx f^\star(x_i)$, the goal is to recover structural properties of $f^\star$.

Classical approaches use structured families such as polynomials, splines, or Fourier expansions, typically characterized by fixed bases.

Neural networks instead define a parametric family

$$\mathcal{F} = \{f_\theta : \theta \in \Theta \subset \mathbb{R}^p\}$$

Learning reduces to empirical risk minimization:

$$\min_{\theta \in \Theta} \frac{1}{n} \sum_{i=1}^{n} \ell(f_\theta(x_i), y_i)$$

Approximation vs Estimation

It is useful to decompose the learning problem into two components:

Approximation: How well can $\mathcal{F}$ represent $f^\star$?

$$\inf_{f \in \mathcal{F}} \|f - f^\star\|$$

Estimation: How accurately can we recover the best $f \in \mathcal{F}$ from finite data?

Neural networks are designed to have:

- Very small approximation error (high expressivity)
- Potentially large estimation error (due to high dimension p)

This tension underlies most theoretical questions in deep learning.

Neural Networks (NN)

A neural network is a parametric function $f_\theta(x)$ that approximates an unknown mapping $x \mapsto y$ via compositions of simple nonlinear transformations.

Core Idea: Input features are successively transformed through layers:

$$x \;\to\; h^{(1)} \;\to\; \cdots \;\to\; h^{(L)} = f_\theta(x)$$

Key Components: Linear maps (W, b), nonlinearities σ, layered composition.

Parameters θ are estimated from data by minimizing a loss function:

$$\min_\theta \sum_{i=1}^{N} \ell\big(y_i, f_\theta(x_i)\big)$$

NNs can be viewed as flexible function approximators with structure determined by architecture.

Neural Networks as Compositions

A neural network is formed by composing simple functions (layers):

$$f_\theta = f_L \circ f_{L-1} \circ \cdots \circ f_1$$

Each layer has the form:

$$f_l(x) = \sigma(W^{(l)}x + b^{(l)})$$

Thus the network can be written explicitly as:

$$f_\theta(x) = \sigma\Big(W^{(L)}\,\sigma\big(\cdots\sigma(W^{(1)}x + b^{(1)})\cdots\big) + b^{(L)}\Big)$$

Interpretation:

- Each layer transforms its input into a new representation
- Composition builds increasingly complex functions from simple units
- Depth corresponds to the number of compositions

The Artificial Neuron

A single neuron defines

$$x \mapsto \sigma(w^T x + b)$$

Here $w \in \mathbb{R}^d$, $b \in \mathbb{R}$, and σ introduces nonlinearity.

Common activations:

$$\text{ReLU}(z) = \max(0, z), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

Without σ, compositions reduce to affine maps:

$$W_2(W_1 x + b_1) + b_2 = (W_2 W_1)x + (W_2 b_1 + b_2)$$

Thus nonlinearity is essential for expressive power.

Role of Nonlinearity in Neural Networks

If $\sigma(z) = z$ (identity), then

$$f(x) = \left(\sum_{m=1}^{M} \beta_m w_m^T \right) x + \sum_{m=1}^{M} \beta_m b_m$$

reduces to a linear model in x

Nonlinear σ enlarges the function class

Activation Scaling:

- $\sigma(sv)$ controls steepness of activation
- Small $|s|$: approximately linear regime
- Large $|s|$: near step-function (hard threshold)

Thus, effective nonlinearity depends on the magnitude of parameters.

Activation Functions

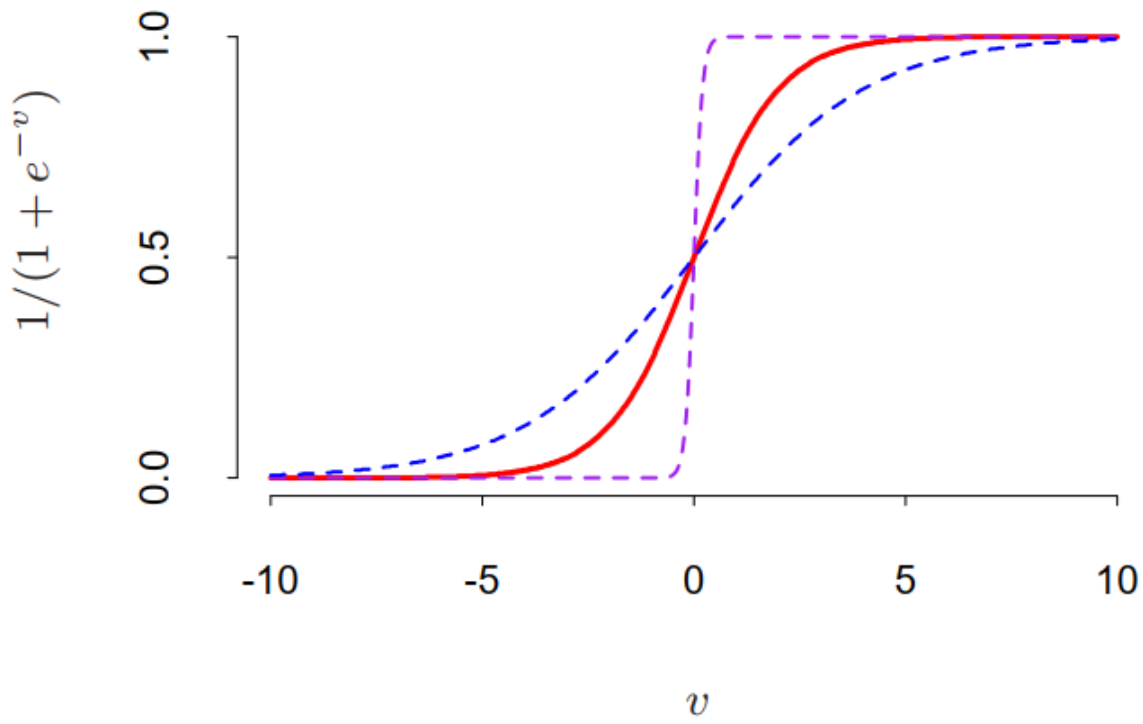


FIGURE 11.3. *Plot of the sigmoid function* $\sigma(v) = 1/(1+\exp(-v))$ *(red curve), commonly used in the hidden layer of a neural network. Included are* $\sigma(sv)$ *for* $s = \frac{1}{2}$ *(blue curve) and* $s = 10$ *(purple curve). The scale parameter* s *controls the activation rate, and we can see that large* s *amounts to a hard activation at* $v = 0$*. Note that* $\sigma(s(v - v_0))$ *shifts the activation threshold from* 0 *to* v_0*.*

From Neuron to Layer

A layer is given by

$$f(x) = \sigma(Wx + b)$$

where $W \in \mathbb{R}^{m \times d}$ and $b \in \mathbb{R}^m$.

This defines a mapping:

$$\mathbb{R}^d \to \mathbb{R}^m$$

Interpretation:

- $Wx + b$: affine transformation (linear projection + shift)
- σ: nonlinear feature extraction

Thus each layer performs a nonlinear transformation of coordinates.

Schematic

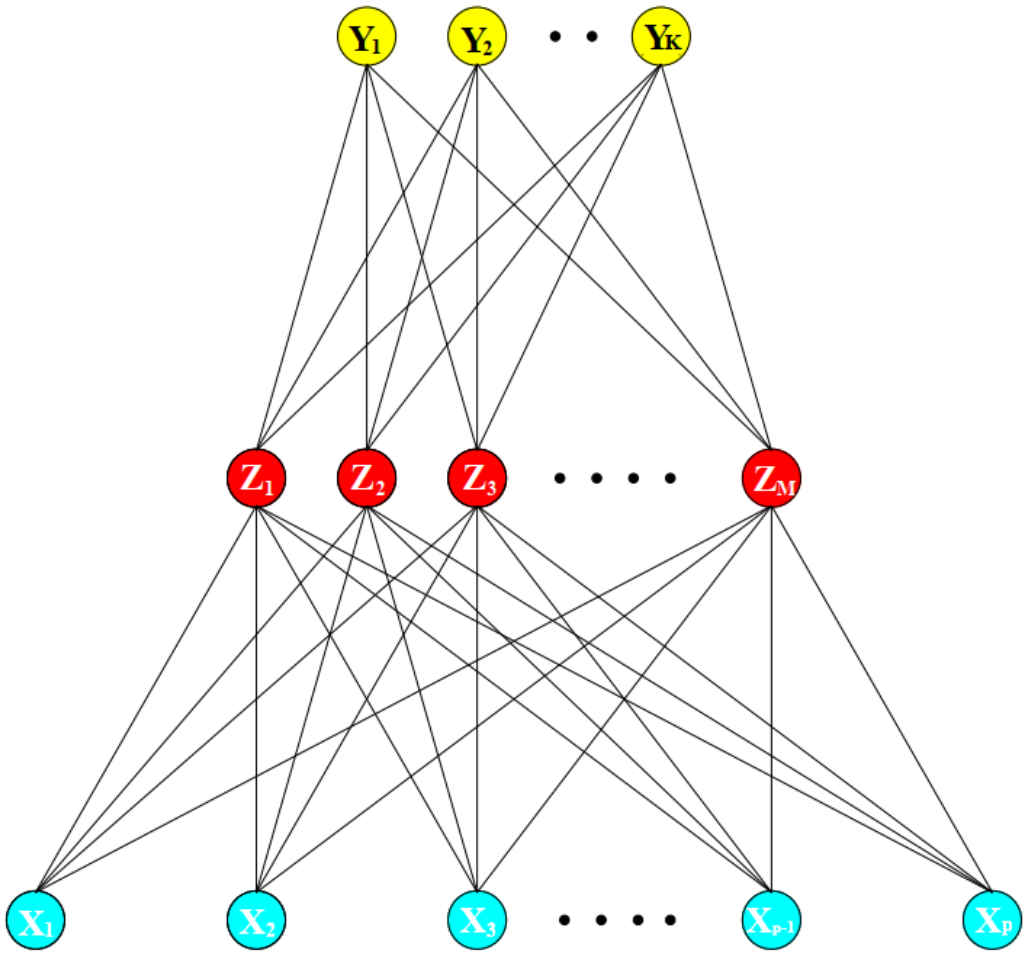


FIGURE 11.2. *Schematic of a single hidden layer, feed-forward neural network.*

Depth vs Width

Consider two regimes:

Shallow networks:

$$f(x) = \sum_{j=1}^{m} a_j \sigma(w_j^T x + b_j)$$

Deep networks:

$$f = f_L \circ \cdots \circ f_1$$

Results show:

- Some functions require exponentially many neurons if shallow
- The same functions admit compact deep representations

Thus depth provides representational efficiency.

Universal Approximation

Statement: If σ is non-polynomial, then for any continuous f on a compact set K,

$$\sup_{x \in K} |f(x) - \sum_{j=1}^{m} a_j \sigma(w_j^T x + b_j)| \to 0$$

However:

- No rate of approximation is guaranteed
- Required width m may be very large

Depth improves approximation rates for structured function classes.

Thus the theorem is qualitative, not algorithmic.

Learning as Optimization

Training solves:

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^{n} \ell(f_\theta(x_i), y_i)$$

This is a non-convex problem in $\mathbb{R}^p$ with $p \gg d$ typically.

Gradient descent:

$$\theta_{t+1} = \theta_t - \eta \nabla \mathcal{L}(\theta_t)$$

Despite non-convexity:

- Gradient-based methods often find low training error solutions in practice
- The final solution depends on initialization and optimization dynamics
- Global minima often exist in overparameterized regimes

Backpropagation as Chain Rule

Forward pass:

$$z^{(l)} = W^{(l)} h^{(l-1)} + b^{(l)}, \quad h^{(l)} = \sigma^{(l)}(z^{(l)})$$

Error signals:

$$\delta^{(l)} := \frac{\partial \ell}{\partial z^{(l)}}$$

$$\delta^{(l)} = \left(W^{(l+1)\,T} \delta^{(l+1)} \right) \odot \sigma^{(l)\,\prime}(z^{(l)})$$

Gradients:

$$\frac{\partial \ell}{\partial W^{(l)}} = \delta^{(l)} (h^{(l-1)})^T, \quad \frac{\partial \ell}{\partial b^{(l)}} = \delta^{(l)}$$

Some Issues in Training Neural Networks

Context:

- Training neural networks involves nonconvex optimization in a highly overparameterized setting
- Stable and effective learning requires careful design choices

Core Challenges:

- Initialization of weights
- Overfitting and regularization
- Scaling of inputs
- Model size selection
- Vanishing gradients
- Multiple minima (nonconvexity)

Initialization of Weights

Small Random Initialization:

- Weights initialized near zero $\Rightarrow$ network initially behaves approximately linear
- Nonlinearity emerges as weights grow during training
- Random small weights break symmetry

Degenerate Cases:

- Zero initialization $\Rightarrow$ symmetry: neurons learn identical features
- Large weights $\Rightarrow$ unstable optimization, poor solutions

Nonconvexity:

- Many local minima
- Solutions depend on initialization

Practice:

- Use multiple restarts or averaging

Overfitting and Regularization

Issue: Excess parameters $\Rightarrow$ model fits noise at global minimum

Early Stopping: Stop training before convergence: acts as implicit shrinkage toward simpler (near-linear) models

Weight Decay:

$$\frac{1}{n}\sum_{i=1}^{n}\ell(f_\theta(x_i), y_i) + \lambda J(\theta), \quad J(\theta) = \sum_{k,m}\beta_{km}^2 + \sum_{m,\ell}\alpha_{m\ell}^2$$

Penalizes large weights and controls model complexity.

Observation:

- Without regularization $\Rightarrow$ overfitting, irregular decision boundary
- With weight decay $\Rightarrow$ smoother, more stable predictions

Example-1

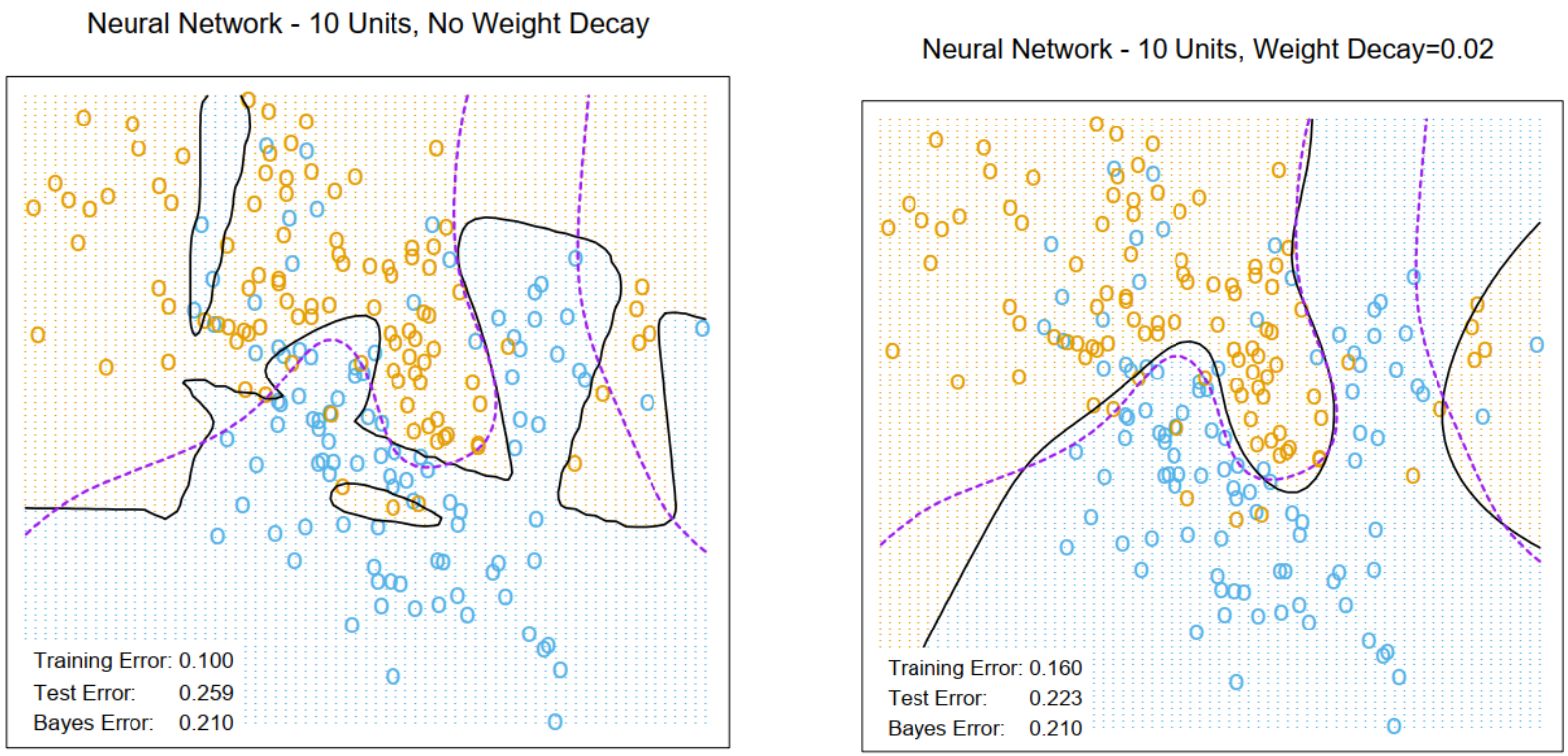


FIGURE 11.4. *A neural network on the mixture example of Chapter 2. The upper panel uses no weight decay, and overfits the training data. The lower panel uses weight decay, and achieves close to the Bayes error rate (broken purple boundary). Both use the softmax activation function and cross-entropy error.*

Example-2

Figure 11.5 shows heat maps of the estimated weights from the training (grayscale versions of these are called *Hinton diagrams*.) We see that weight decay has dampened the weights in both layers: the resulting weights are spread fairly evenly over the ten hidden units

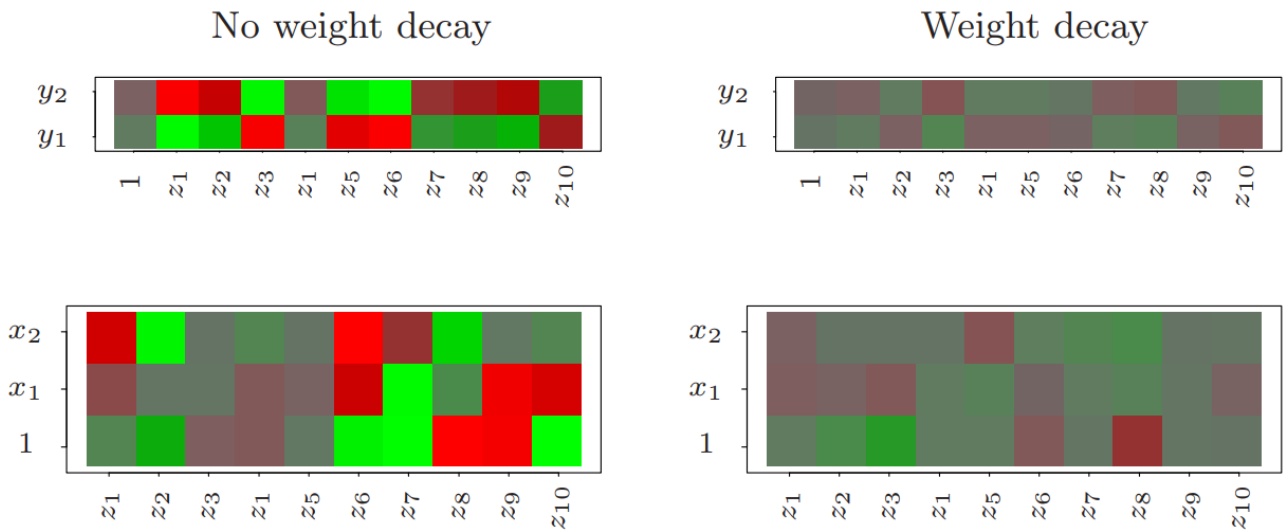


FIGURE 11.5. *Heat maps of the estimated weights from the training of neural networks from Figure 11.4. The display ranges from bright green (negative) to bright red (positive).*

Scaling and Model Size

Standardize inputs: $x_j \leftarrow \frac{x_j - \mu_j}{\sigma_j}$.

Because input scale determines effective scale of weights and unequal scaling distorts regularization.

Practical Choice: Initialize weights in a fixed range (e.g., uniform)

Model size (Hidden Units):

- Too few $\Rightarrow$ insufficient flexibility
- Too many $\Rightarrow$ handled via regularization

Practical choice: Choose a moderately large number of units and let regularization control effective complexity.

Depth enables:

- Multiple layers $\Rightarrow$ hierarchical feature extraction
- Captures structure at different levels of abstraction

Vanishing/Exploding Gradients

$$\frac{\partial \mathcal{L}}{\partial h^{(0)}} = W^{(1)T} D^{(1)} W^{(2)T} D^{(2)} \dots W^{(L)T} D^{(L)}$$

where

$$D^{(l)} = \text{diag}(\sigma'(z^{(l)}))$$

- Exponential decay or growth across layers
- Severe with depth and saturating activations
- Early layers learn slowly or not at all

Nonconvex Optimization:

- Many local minima and saddle points
- Plateaus $\Rightarrow$ slow progress
- Sensitive to initialization and learning rate

Stabilization Techniques

Residual Connections:

$$h^{(l+1)} = h^{(l)} + F(h^{(l)})$$

- Direct gradient paths
- Enables deep networks

Normalization:

$$\hat{x} = \frac{x - \mu}{\sigma}$$

- Stabilizes layer inputs
- Faster convergence
- Faster and more stable training

Limits of the Classical Bias–Variance View

Classical View:

- Tradeoff: low bias $\leftrightarrow$ high variance
- Larger models $\Rightarrow$ worse generalization

Modern Observations (Deep Learning):

- $p \gg n$: models interpolate yet generalize well
- Double descent: test error decreases again beyond interpolation
- Parameter count $\neq$ effective complexity

Why the Mismatch:

- Many global minima (nonconvexity)
- Optimization selects specific solutions (implicit regularization)
- Architecture constrains learned functions

Takeaway: Bias–variance is valid but not predictive in overparameterized regimes.

ZIP Code Networks: Problem Setup

Input–Output Structure:

- Input: $x \in \mathbb{R}^{256}$ (flattened 16×16 image)
- Output: $f_k(x),\ k = 0, \ldots, 9$

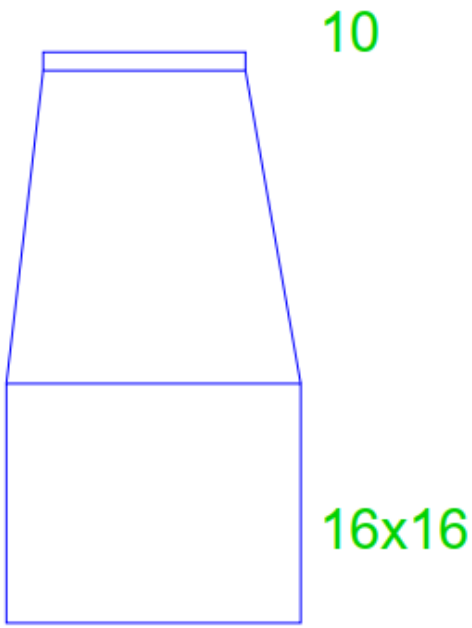
$$f_k(x) = \frac{e^{z_k(x)}}{\sum_{j=0}^{9} e^{z_j(x)}}, \quad \sum_{k=0}^{9} f_k(x) = 1$$

- $f_k(x)$ estimates $P(Y = k \mid x)$

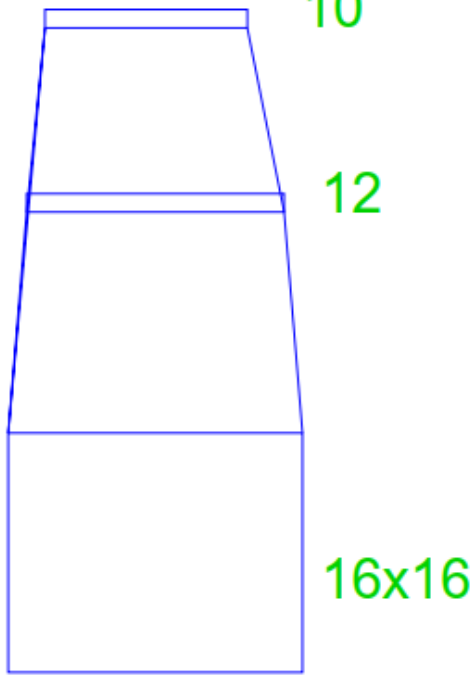
Common Setup:

- Softmax output units
- Sum-of-squares loss
- Overparameterized: training error ≈ 0

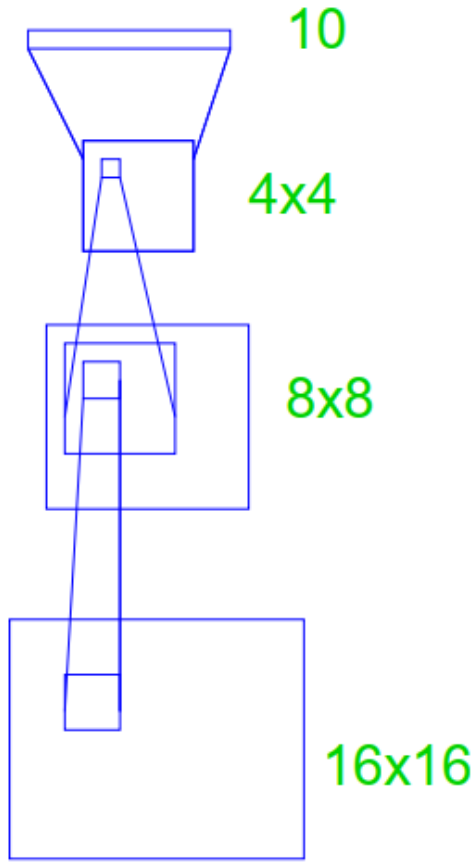
Architectures-1



Net-1



Net-2



Net-3
Local Connectivity

ZIP Networks: Net-1, Net-2, Net-3

Net-1 (Linear Model):

- No hidden layer: $\mathbb{R}^{256} \to \mathbb{R}^{10}$
- Equivalent to multinomial logistic regression
- Linear decision boundaries, limited expressivity

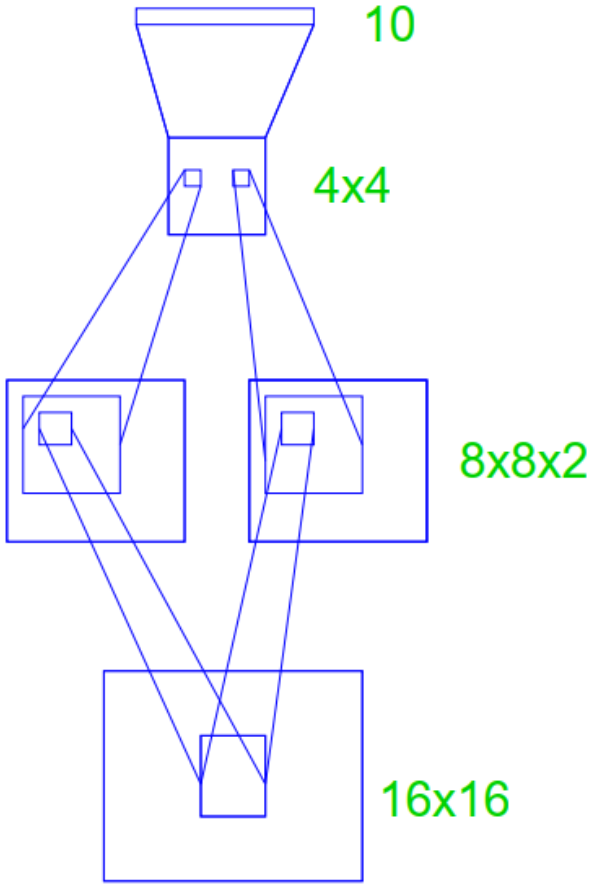
Net-2 (Fully Connected ANN):

- One hidden layer with 12 units
- Dense connectivity: global interactions between pixels
- Increased flexibility with large number of parameters

Net-3 (Local Connectivity):

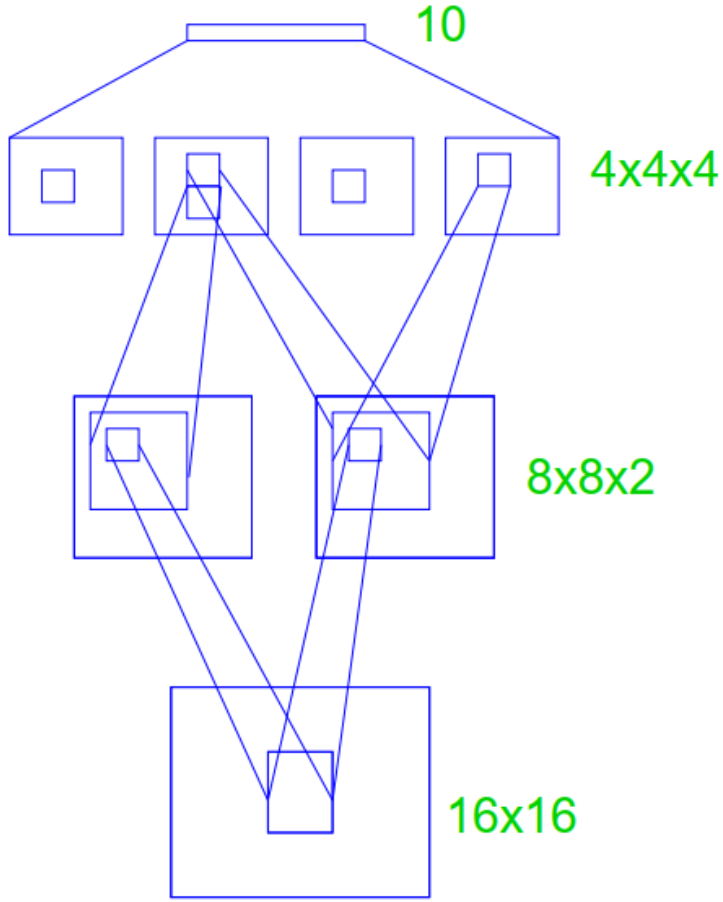
- Two hidden layers with spatial structure ($16 \times 16 \to 8 \times 8$)
- Local receptive fields (3×3, then 5×5)
- Extracts local features with fewer effective parameters

Architectures-2



Net-4

Shared Weights



Net-5

ZIP Networks: Net-4, Net-5

Net-4 (Local + Weight Sharing):

- Two hidden layers with local connectivity
- First layer: feature maps with shared weights
- Same filter applied across spatial locations
- Translation-equivariant feature extraction

Net-5 (Hierarchical Weight Sharing):

- Multiple feature maps with sharing across layers
- Shared filters at both lower and higher levels
- Builds hierarchical features from local patterns

Key Insight:

- Increasing structural constraints $\Rightarrow$ fewer parameters + better generalization

From ZIP Networks to CNNs

- Net-3: local connectivity
- Net-4/5: local connectivity + weight sharing

$$W^{(l)} \equiv \text{convolution operator}$$

- These architectures implement convolution implicitly
- CNNs formalize this structure

Deep Neural Networks: Structure

Model:

$$h^{(0)} = x, \quad h^{(l)} = \sigma(W^{(l)} h^{(l-1)} + b^{(l)}), \quad f(x) = h^{(L)}$$

Depth:

- Multiple hidden layers ($L \gg 1$)
- Composition of nonlinear transformations

Interpretation:

- Each layer builds a new representation of the input
- Higher layers encode more abstract structure

Expressivity of Depth

Compositional Form:

$$f(x) = f_L \circ f_{L-1} \circ \cdots \circ f_1(x)$$

Key Effect:

- Depth increases expressive power beyond shallow models
- Complex functions represented with fewer units

Qualitative Result:

- Number of linear regions grows rapidly with depth
- Efficient representation of hierarchical patterns

Convolutional Neural Networks (CNN): Motivation

Setting:

- Input has spatial structure (e.g., images 16×16)
- Nearby pixels are more strongly related than distant ones

Limitation of Fully Connected Layers:

- Ignore spatial locality
- Large number of parameters

Key Idea:

- Restrict connectivity to local neighborhoods
- Share weights across spatial locations

$$h^{(l)} = \sigma(\mathcal{W}^{(l)} h^{(l-1)})$$

Convolution Operation

Discrete Convolution:

$$Y_{i,j,k} = \sum_{u,v,c} X_{i+u,j+v,c} \, K_{u,v,c,k}$$

Interpretation:

- K is a local filter (kernel). K defines a shared linear operator (Toeplitz structure)
- Same filter applied at all spatial locations
- Produces feature maps

Structure:

- Sparse connectivity (local receptive fields)
- Parameter sharing across (i, j)

Feature Maps and Weight Sharing

Feature Map:

$$Y_{i,j}^{(k)} = \sum_{c} (X^{(c)} * K^{(k,c)})_{i,j}$$

Properties:

- Each map detects a specific pattern (edge, texture, etc.)
- All spatial locations use identical weights

Implication:

- Translation equivariance:

$$T_a(X) \Rightarrow T_a(Y)$$

- Significant reduction in number of parameters

Pooling and Downsampling

Pooling Operation:

$$Y_{i,j,c} = \max_{(u,v)\in\mathcal{R}} X_{i+u,j+v,c}$$

Role:

- Reduces spatial resolution
- Aggregates local information

Effect:

- Improves invariance to small translations
- Controls model complexity

CNN Architecture

Layer Composition:

$$X \to \text{Conv} \to \sigma \to \text{Pool} \to \cdots$$

$$f(x) = \text{FC}(\phi_{\text{conv}}(x))$$

Hierarchy:

- Lower layers: local, simple features
- Higher layers: global, abstract features

Key Principle:

- Structured constraints $\Rightarrow$ efficient representation + better generalization